Cahier de Charges

Application de Suivi de Lecture et de Recommandation de Livres — **BookTracker**

Information	Détail
Projet	Projet de Fin d'Année (PFA)
Version	1.0
Date	19 Février 2026
Auteur	Mohamed Nacer Hammami
Durée	2 mois (8 semaines)
Méthodologie	Scrum (Agile) — sprints de 2 semaines
Architecture	Microservices (4 services + API Gateway)
Déploiement	Docker Compose (local)

Table des Matières

1. Contexte et Objectifs du Projet
2. Périmètre Fonctionnel
3. Architecture Technique
4. Choix Technologiques
5. Modélisation de la Base de Données
6. Sécurité
7. Approche Scrum — Organisation du Projet
8. Product Backlog
9. Planning des Sprints
10. Exigences Non Fonctionnelles
11. Diagrammes UML
12. Maquettes UI/UX
13. Gestion des Risques
14. Livrables
15. Annexes

1. Contexte et Objectifs du Projet

1.1 Contexte

Les lecteurs font face à plusieurs défis du quotidien :

- **Difficulté à organiser** leurs lectures en cours, terminées ou planifiées.

- **Manque de recommandations** adaptées à leurs goûts.
- **Absence d'un outil simple** pour suivre leur progression et leurs statistiques de lecture.

BookTracker propose une application web permettant à chaque lecteur de gérer sa bibliothèque personnelle, suivre sa progression, noter les livres lus et recevoir des recommandations simples.

1.2 Objectifs

#	Objectif	Description
O1	**Suivi de lecture**	Gérer une bibliothèque personnelle (À lire, En cours, Terminé)
O2	**Catalogue de livres**	Rechercher et consulter des fiches livres enrichies via Google Books API
O3	**Avis et Notes**	Permettre aux lecteurs de noter et commenter les livres
O4	**Recommandations simples**	Suggérer des livres basés sur les genres préférés de l'utilisateur
O5	**Statistiques**	Afficher des métriques de lecture (livres lus, pages totales, répartition par genre)
O6	**Sécurité**	Authentification sécurisée (JWT) et protection des données

> **Périmètre PFA :** L'application adopte une **architecture microservices légère** (4 services + API Gateway) déployée via Docker Compose. Les fonctionnalités avancées (filtrage collaboratif, déploiement cloud AWS, Kubernetes, Kafka/RabbitMQ, ELK Stack) sont explicitement hors périmètre et constituent des évolutions futures.

1.3 Public Cible

- Lecteurs réguliers souhaitant organiser leur suivi de lecture
- Étudiants passionnés de lecture

---

2. Périmètre Fonctionnel

2.1 Acteurs du Système

Acteur	Description
Visiteur	Utilisateur non authentifié — peut consulter le catalogue et créer un compte
Lecteur	Utilisateur authentifié — accède à toutes les fonctionnalités de suivi
Administrateur	Gère le catalogue de livres et les utilisateurs

2.2 Modules Fonctionnels

Module 1 — Gestion des Utilisateurs

ID	User Story	Priorité

ID	User Story	Priorité
US-01	En tant que visiteur, je veux m'inscrire avec email et mot de passe	Must
US-02	En tant qu'utilisateur, je veux me connecter de manière sécurisée (JWT)	Must
US-03	En tant qu'utilisateur, je veux réinitialiser mon mot de passe par email	Must
US-04	En tant qu'utilisateur, je veux consulter et modifier mon profil (avatar, bio)	Should
US-05	En tant qu'admin, je veux activer/désactiver des comptes utilisateurs	Should

Module 2 — Catalogue de Livres

ID	User Story	Priorité
US-06	En tant qu'utilisateur, je veux rechercher des livres par titre, auteur ou genre	Must
US-07	En tant qu'utilisateur, je veux consulter la fiche détaillée d'un livre (résumé, couverture, note moyenne)	Must
US-08	En tant qu'admin, je veux ajouter, modifier et supprimer des livres dans le catalogue	Must
US-09	En tant que système, je veux enrichir les fiches livres via l'API Google Books	Should

Module 3 — Suivi de Lecture (Reading Tracker)

ID	User Story	Priorité
US-10	En tant que lecteur, je veux ajouter un livre à ma bibliothèque avec un statut (À lire / En cours / Terminé)	Must
US-11	En tant que lecteur, je veux mettre à jour ma progression (page actuelle)	Must
US-12	En tant que lecteur, je veux voir l'historique de mes lectures avec dates de début et fin	Must
US-13	En tant que lecteur, je veux définir un objectif de lecture annuel	Should

Module 4 — Avis et Notes

ID	User Story	Priorité
US-14	En tant que lecteur, je veux noter un livre (1 à 5 étoiles)	Must
US-15	En tant que lecteur, je veux rédiger une critique sur un livre	Must
US-16	En tant qu'admin, je veux supprimer les avis inappropriés	Should

Module 5 — Recommandations Simples

ID	User Story	Priorité
US-17	En tant que lecteur, je veux recevoir des recommandations basées sur mes genres préférés	Must
US-18	En tant qu'utilisateur, je veux voir les livres les mieux notés sur la plateforme	Should

Module 6 — Statistiques

ID	User Story	Priorité
US-19	En tant que lecteur, je veux voir mes statistiques (livres lus, pages totales, genre favori)	Must
US-20	En tant que lecteur, je veux visualiser ma progression vers mon objectif annuel	Should
US-21	En tant que lecteur, je veux voir la répartition de mes lectures par genre (graphique)	Should

3. Architecture Technique

3.1 Architecture Globale — Microservices (PFA)

L'architecture retenue est un **ensemble de 4 microservices Spring Boot** exposés derrière un **API Gateway** (Spring Cloud Gateway), avec un frontend **React.js** séparé. L'ensemble est déployé via **Docker Compose** en local.

> **Choix d'architecture :** Pour respecter le délai de 2 mois, la décomposition est volontairement limitée à **4 services métier** cohérents. Les patterns avancés (Eureka, Kafka, ELK, Kubernetes) sont hors périmètre — ils constituent les évolutions naturelles vers un PFE.

```
┌──────────────────────────────────────────────────────────────────┐
│                   FRONTEND (React.js + Vite)                     │
│                        Port : 3000                               │
└──────────────────────────────────────────────────────────────────┘
                                │
                                │ HTTP (REST)
                                ▼
┌──────────────────────────────────────────────────────────────────┐
│              API GATEWAY – Spring Cloud Gateway                  │
│              Port : 8080                                         │
│    ┌──────────────────────────────────────────────────────┐      │
│    │  Routing │ JWT Validation │ CORS │ Rate Limiting     │      │
│    └──────────────────────────────────────────────────────┘      │
└──────────────────────────────────────────────────────────────────┘
        │              │              │              │
        ▼              ▼              ▼              ▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐ ┌──────────────────┐
│ auth-service │ │ book-service │ │tracker-service│ │  reco-service    │
│   Port 8081  │ │  Port 8082   │ │  Port 8083   │ │   Port 8084      │
│              │ │              │ │              │ │                  │
```

```
│  Auth/Users   │ │  Catalogue    │ │  Tracking     │ │  Recommandations │
│  JWT/BCrypt   │ │  G.Books API  │ │  Reviews      │ │  Statistiques    │
└───────────────┘ └───────────────┘ └───────────────┘ └──────────────────┘
        │                 │                 │                   │
        ▼                 ▼                 ▼                   ▼
┌─────────────┐     ┌─────────────┐     ┌─────────────┐     ┌─────────────┐
│ PostgreSQL  │     │ PostgreSQL  │     │ PostgreSQL  │     │ PostgreSQL  │
│ auth_db     │     │ book_db     │     │ tracker_db  │     │ + Redis     │
└─────────────┘     └─────────────┘     └─────────────┘     └─────────────┘
```

3.2 Description des Services

Service	Port	Responsabilité
api-gateway	8080	Point d'entrée unique — routing, validation JWT, CORS
auth-service	8081	Inscription, connexion, gestion profil utilisateur
book-service	8082	Catalogue de livres, CRUD admin, intégration Google Books API
tracker-service	8083	Suivi de lecture (UserBook), historique, avis et notes
reco-service	8084	Recommandations par genre, statistiques de lecture, cache Redis

3.3 Communication Inter-Services

Type	Technologie	Usage
Synchrone	REST via OpenFeign	reco-service → tracker-service (récupérer genres lus)
Synchrone	REST via OpenFeign	tracker-service → book-service (vérifier existence livre)
Résolution DNS	Docker Compose DNS	Noms de service utilisés comme hostnames (ex: `http://book-service:8082`)

> **Pas de message broker** pour ce PFA — la communication asynchrone (RabbitMQ/Kafka) est une évolution future.

3.4 Architecture en Couches (par Microservice)

```
┌──────────────────────────────────────┐
│          Controller Layer            │  ← REST API (@RestController)
├──────────────────────────────────────┤
│          DTO / Mapper Layer          │  ← MapStruct
├──────────────────────────────────────┤
│          Service Layer               │  ← Business Logic (@Service)
├──────────────────────────────────────┤
│          Repository Layer            │  ← Spring Data JPA (@Repository)
├──────────────────────────────────────┤
```

```
│        Entity / Model Layer          │  ← JPA Entities (@Entity)
└──────────────────────────────────────┘
```

---

4. Choix Technologiques

4.1 Stack Technique

Couche	Technologie	Justification
Backend	Java 17 + Spring Boot 3.x	Robustesse, écosystème riche, standard industrie
API Gateway	Spring Cloud Gateway	Routing, JWT validation, CORS centralisés
Communication inter-services	Spring Cloud OpenFeign	Appels REST déclaratifs entre services
Frontend	React.js 18 + TypeScript	SPA performante, typage fort
UI Framework	Tailwind CSS + shadcn/ui	Design moderne, composants accessibles
Base de données	PostgreSQL 16 (1 DB par service)	Isolation des données par domaine métier
Cache	Redis 7	Cache recommandations + sessions JWT
API Documentation	SpringDoc OpenAPI (Swagger)	Documentation auto-générée par service
Tests	JUnit 5 + Mockito	Tests unitaires et d'intégration
Build Backend	Maven (multi-modules)	Gestion centralisée des dépendances
Build Frontend	Vite	Build rapide
Conteneurisation	Docker + Docker Compose	Orchestration locale de tous les services
Sécurité	Spring Security + JWT	Auth stateless — token validé au niveau Gateway
API Externe	Google Books API	Enrichissement des fiches livres (book-service)

4.2 Structure du Projet Frontend (React.js)

```
src/
├── api/              # Axios instances et appels API
├── components/       # Composants réutilisables
│   ├── layout/       # Header, Sidebar, Footer
│   ├── books/        # BookCard, BookDetail
│   └── reading/      # ReadingProgress, StatusBadge
├── pages/            # Pages principales
```

```
│   ├── Home.tsx
│   ├── Login.tsx
│   ├── Register.tsx
│   ├── Dashboard.tsx
│   ├── BookCatalog.tsx
│   ├── BookDetail.tsx
│   ├── MyLibrary.tsx
│   ├── Statistics.tsx
│   └── AdminPanel.tsx
├── hooks/              # Custom hooks
├── store/              # Zustand (state management)
├── types/              # TypeScript interfaces
└── App.tsx
```

---

5. Modélisation de la Base de Données

> **Principe microservices :** Chaque service possède **sa propre base de données PostgreSQL** (Database per Service pattern). Les services ne partagent pas de tables — ils communiquent via API REST.

Service	Base de données	Tables principales
auth-service	`auth_db`	users
book-service	`book_db`	books, authors, genres, book_genre
tracker-service	`tracker_db`	user_books, reviews
reco-service	`reco_db`	(requêtes vers tracker-service + Redis cache)

5.1 Modèle Conceptuel de Données (MCD)

```
┌──────────────┐        ┌──────────────────┐        ┌──────────────┐
│   USER       │        │    USER_BOOK     │        │     BOOK     │
├──────────────┤        ├──────────────────┤        ├──────────────┤
│ id (PK)      │──────▶│ id (PK)          │◀──────│ id (PK)      │
│ email        │        │ user_id (FK)     │        │ title        │
│ password     │        │ book_id (FK)     │        │ isbn         │
│ first_name   │        │ status           │        │ description  │
│ last_name    │        │ current_page     │        │ cover_url    │
│ avatar_url   │        │ start_date       │        │ page_count   │
│ bio          │        │ end_date         │        │ publish_date │
│ role         │        │ created_at       │        │ language     │
│ reading_goal │        └──────────────────┘        │ avg_rating   │
│ is_active    │                                     │ author       │
│ created_at   │        ┌──────────────────┐        │ publisher    │
└──────────────┘        │     REVIEW       │        └──────────────┘
       │                ├──────────────────┤                │
       └──────────────▶│ id (PK)          │◀──────────────┘
                        │ user_id (FK)     │
                        │ book_id (FK)     │        ┌──────────────┐
                        │ content          │        │    GENRE     │
```

```
                          │ rating              │        ├──────────────┤
                          │ created_at          │        │ id (PK)      │
                          └─────────────────────┘        │ name         │
                                                         └──────┬───────┘
                                                                │  N:M
                                                         ┌──────┴───────┐
                                                         │  BOOK_GENRE  │
                                                         └──────────────┘
```

5.2 Dictionnaire de Données

Entité	Attribut	Type	Contrainte	Description
USER	id	UUID	PK, NOT NULL	Identifiant unique
USER	email	VARCHAR(255)	UNIQUE, NOT NULL	Adresse email
USER	password	VARCHAR(255)	NOT NULL	Hash BCrypt
USER	role	ENUM	NOT NULL	READER, ADMIN
USER	reading_goal	INTEGER	NULL	Objectif annuel de livres
BOOK	isbn	VARCHAR(13)	UNIQUE	Code ISBN-13
BOOK	page_count	INTEGER	NOT NULL	Nombre de pages
USER_BOOK	status	ENUM	NOT NULL	TO_READ, READING, FINISHED
USER_BOOK	current_page	INTEGER	DEFAULT 0	Page actuelle
REVIEW	rating	DECIMAL(2,1)	1.0 – 5.0	Note du livre

6. Sécurité

6.1 Stratégie de Sécurité

Couche	Mécanisme
Authentification	JWT (JSON Web Token) — stateless
Hachage mot de passe	BCrypt (strength=12)
Autorisation	RBAC via Spring Security (`@PreAuthorize`)
Protection données	HTTPS (TLS), validation des entrées
CORS	Restriction aux origines frontend uniquement

6.2 Flux d'Authentification JWT

```
Client              API Gateway          auth-service         book-service
  │                     │                    │                    │
```

```
    |─ POST /auth/login ▶|                    |                    |
    |                    |─ Route ──────────▶|                    |
    |                    |                    |── Vérif. BCrypt    |
    |                    |                    |── Génération JWT   |
    |                    |◀── 200 {token} ──|                    |
    |◀── 200 {token} ──|                    |                    |
    |                    |                    |                    |
    |─ GET /books ──────▶|                    |                    |
    |   Bearer <JWT>     |── Valider JWT      |                    |
    |                    |── Extraire rôle    |                    |
    |                    |── Route ──────────────────────────────▶|
    |                    |◀──────────────────────── 200 {books[]} ─|
    |◀── 200 {books[]} ─|                    |                    |
```

6.3 Matrice d'Autorisation (RBAC)

Ressource	Visiteur	Lecteur	Admin
Consulter catalogue	☑	☑	☑
Rechercher livres	☑	☑	☑
Inscription / Connexion	☑	—	—
Gérer bibliothèque	✕	☑	☑
Suivi de lecture	✕	☑	☑
Écrire un avis	✕	☑	☑
Voir recommandations	✕	☑	☑
Voir statistiques	✕	☑	☑
CRUD livres	✕	✕	☑
Gérer utilisateurs	✕	✕	☑
Supprimer avis	✕	✕	☑

6.4 Protection OWASP (essentiels)

Vulnérabilité	Mesure
Injection SQL	JPA Criteria API, requêtes paramétrées
Auth Failures	Rate limiting sur login, token expiration
Broken Access Control	RBAC via annotations Spring Security
Cryptographic Failures	BCrypt, HTTPS/TLS
Insecure Config	Variables d'environnement, pas de secrets en dur

---

7. Approche Scrum — Organisation du Projet

7.1 Rôles Scrum

Rôle	Responsable
Product Owner	Mohamed Nacer Hammami
Scrum Master	Mohamed Nacer Hammami
Développeur	Mohamed Nacer Hammami

7.2 Paramètres du Projet

Paramètre	Valeur
Durée totale	**8 semaines**
Durée du sprint	**2 semaines**
Nombre de sprints	**4 sprints**
Vélocité estimée	**20–25 story points / sprint**
Definition of Done	Code compilable, testé (basique), fonctionnalité démontrée
Outil de gestion	GitHub Projects

7.3 Cérémonies Scrum (adaptées — projet solo)

Cérémonie	Fréquence	Durée
Sprint Planning	Début de sprint	1h
Daily (auto-journal)	Quotidien	10 min
Sprint Review	Fin de sprint	30 min
Rétrospective	Fin de sprint	20 min

---

8. Product Backlog

8.1 Estimation (Story Points — Fibonacci)

Points	Complexité	Exemple
1	Triviale	Changement de texte, ajout d'un champ
2	Simple	CRUD simple
3	Modérée	Fonctionnalité avec logique métier
5	Complexe	Intégration API externe, logique avancée
8	Très complexe	Algorithme de recommandation

8.2 Backlog Priorisé

ID	User Story	Module	Points	Priorité	Sprint
US-01	Inscription email + mot de passe	Auth	3	Must	S1
US-02	Connexion sécurisée JWT	Auth	3	Must	S1
US-03	Réinitialisation mot de passe	Auth	2	Must	S1
US-06	Recherche de livres	Catalogue	3	Must	S1
US-07	Fiche détaillée livre	Catalogue	2	Must	S1
US-08	CRUD livres (admin)	Catalogue	3	Must	S1
US-09	Intégration Google Books API	Catalogue	5	Should	S2
US-10	Ajout livre + statut biblio	Tracking	3	Must	S2
US-11	Mise à jour progression lecture	Tracking	2	Must	S2
US-12	Historique des lectures	Tracking	2	Must	S2
US-14	Notation livre (étoiles)	Reviews	2	Must	S2
US-15	Rédaction d'avis	Reviews	3	Must	S2
US-04	Profil utilisateur	Auth	2	Should	S3
US-13	Objectif de lecture annuel	Tracking	2	Should	S3
US-17	Recommandations par genre	Reco	5	Must	S3
US-18	Livres les mieux notés	Reco	2	Should	S3
US-19	Statistiques de lecture	Stats	3	Must	S3
US-20	Progression objectif annuel	Stats	2	Should	S3
US-21	Graphique répartition genres	Stats	3	Should	S4
US-05	Gestion utilisateurs (admin)	Auth	2	Should	S4
US-16	Modération avis (admin)	Reviews	2	Should	S4
—	Tests unitaires (80% coverage)	QA	5	Must	S4
—	Docker Compose + déploiement local	DevOps	3	Must	S4
—	Documentation API Swagger	Docs	2	Must	S4

---

9. Planning des Sprints

Sprint 0 — Setup & Architecture (Semaine 0 — pré-sprint)

- ☐ Initialisation projet Maven multi-modules (parent POM + 5 modules : gateway, auth, book, tracker, reco)
- ☐ Initialisation projet React.js (Vite + TypeScript)
- ☐ Configuration Docker Compose (5 services + 4 PostgreSQL + Redis)

- ☐ Configuration Spring Cloud Gateway (routes de base)
- ☐ Création du repository Git, branches (main, develop, feature/*)
- ☐ Setup CI de base (GitHub Actions — build + tests)

Sprint 1 — auth-service & book-service (Semaines 1-2)

Objectif : L'utilisateur peut s'inscrire, se connecter et parcourir le catalogue de livres via l'API Gateway.

Tâche	Service	Description	Points
T1.1	auth-service	Entité User + Repository + Migration Flyway	2
T1.2	auth-service	Service Auth (JWT + BCrypt) + endpoints login/register	3
T1.3	auth-service	Réinitialisation mot de passe (email)	2
T1.4	api-gateway	Configuration des routes + validation JWT centralisée	3
T1.5	book-service	Entité Book + Author + Genre + Repository + Flyway	2
T1.6	book-service	API REST catalogue (GET, search) + CRUD admin	3
T1.7	Frontend	Pages Login / Register (React)	3
T1.8	Frontend	Page Catalogue + Recherche (React)	3
Total			**21**

Sprint 2 — tracker-service & enrichissement book-service (Semaines 3-4)

Objectif : Le lecteur peut gérer sa bibliothèque et ses avis ; le catalogue est enrichi via Google Books.

Tâche	Service	Description	Points
T2.1	book-service	Intégration Google Books API (OpenFeign externe)	5
T2.2	tracker-service	Entité UserBook + API suivi de lecture + Flyway	3
T2.3	tracker-service	Mise à jour progression (page actuelle)	2
T2.4	tracker-service	Historique des lectures	2
T2.5	tracker-service	Entité Review + API avis et notes	3
T2.6	Frontend	Page Ma Bibliothèque (React)	3
T2.7	Frontend	Composant notation + avis (React)	3
Total			**21**

Sprint 3 — reco-service & Statistiques (Semaines 5-6)

Objectif : Recommandations par genre (content-based), tableau de bord statistiques, profil utilisateur.

Tâche	Service	Description	Points
T3.1	auth-service	Profil utilisateur — endpoint GET/PUT /users/me	2

Tâche	Service	Description	Points
T3.2	tracker-service	Objectif de lecture annuel	2
T3.3	reco-service	Appel OpenFeign → tracker-service (genres lus)	2
T3.4	reco-service	Algorithme recommandation content-based (genres préférés)	5
T3.5	reco-service	Statistiques de lecture (livres lus, pages, genre favori)	3
T3.6	reco-service	Cache Redis des recommandations et stats	2
T3.7	Frontend	Page Recommandations (React)	2
T3.8	Frontend	Dashboard statistiques (React + Recharts)	3
Total			**21**

Sprint 4 — Finalisation, Tests & Qualité (Semaines 7-8)

Objectif : Admin panel, tests, documentation Swagger par service, Docker Compose complet, rapport.

Tâche	Service	Description	Points
T4.1	Frontend	Graphique répartition genres (Recharts)	3
T4.2	auth-service + Frontend	Panel admin — gestion utilisateurs et modération avis	2
T4.3	Tous	Tests unitaires par service (couverture > 70%)	5
T4.4	Tous	Documentation API Swagger/OpenAPI (1 Swagger par service)	2
T4.5	DevOps	Docker Compose complet — health checks, dépendances entre services	3
T4.6	Frontend + All	Corrections bugs, amélioration UX	3
T4.7	—	Rapport PFA + préparation soutenance	2
Total			**20**

10. Exigences Non Fonctionnelles

10.1 Performance

Métrique	Objectif
Temps de réponse API	< 500ms (P95)
Temps de chargement page	< 3 secondes
Disponibilité	Environnement de développement local

10.2 Qualité du Code

Critère	Objectif
Couverture de tests	> 70% (services critiques)
Documentation API	100% des endpoints documentés via Swagger
Code style	Checkstyle / ESLint actifs

10.3 Accessibilité & UX

Critère	Standard
Responsive Design	Mobile et Desktop (min. 768px)
Navigateurs supportés	Chrome, Firefox (dernières versions)
Langue	Français

---

11. Diagrammes UML

11.1 Diagramme de Cas d'Utilisation

```
                    ┌──────────────────────────────────────────┐
                    │           BookTracker System             │
                    │                                          │
┌──────────┐        │  ┌──────────────────────────────┐        │
│ Visiteur │──────▶│  │  Consulter / Rechercher livres│       │
└──────────┘   │    │  └──────────────────────────────┘        │
               └───▶│  ┌──────────────────────────────┐        │
                    │  │  S'inscrire / Se connecter    │       │
                    │  └──────────────────────────────┘        │
                    │                                          │
┌──────────┐        │  ┌──────────────────────────────┐        │
│ Lecteur  │──────▶│  │  Gérer ma bibliothèque         │       │
└──────────┘   │    │  └──────────────────────────────┘        │
               ├───▶│  ┌──────────────────────────────┐        │
               │    │  │  Suivre progression lecture    │       │
               │    │  └──────────────────────────────┘        │
               ├───▶│  ┌──────────────────────────────┐        │
               │    │  │  Écrire avis / Noter un livre  │       │
               │    │  └──────────────────────────────┘        │
               ├───▶│  ┌──────────────────────────────┐        │
               │    │  │  Consulter recommandations     │       │
               │    │  └──────────────────────────────┘        │
               └───▶│  ┌──────────────────────────────┐        │
                    │  │  Voir mes statistiques         │       │
                    │  └──────────────────────────────┘        │
                    │                                          │
┌──────────┐        │  ┌──────────────────────────────┐        │
│  Admin   │──────▶│  │  Gérer catalogue (CRUD livres) │       │
└──────────┘   │    │  └──────────────────────────────┘        │
               └───▶│  ┌──────────────────────────────┐        │
```

```
                      │  │  Gérer utilisateurs / avis     │       │
                      │  └──────────────────────────────┘       │
                      │                                         │
                      │                                         │
  ┌──────────┐        │  ┌──────────────────────────────┐       │
  │ Google   │◁──────▶│  │  Enrichir données livres      │       │
  │ Books API│        │  └──────────────────────────────┘       │
  └──────────┘        └─────────────────────────────────────────┘
```

11.2 Diagramme de Séquence — Ajout d'un Livre à la Bibliothèque

```
Lecteur    Frontend    API Gateway    tracker-service    book-service
tracker_db
  │            │             │               │                │             │
  │─ Clic ────▶│             │               │                │             │
  │            │─ POST       │               │                │             │
  │            │  /tracking ▶│               │                │             │
  │            │  Bearer JWT │─ Valider JWT  │                │             │
  │            │             │─ Router ─────▶│                │             │
  │            │             │               │─ GET /books/{id}▶            │
  │            │             │               │◀─ 200 {book} ──│             │
  │            │             │               │─ INSERT user_book ──────────▶│
  │            │             │               │◀────────────────────────────│
  │            │             │◀─ 201 Created ─│                │             │
  │            │◀─ 201 ──────│               │                │             │
  │◀─ "Ajouté" ─│            │               │                │             │
```

11.3 Diagramme de Classes Simplifié

```
┌──────────────┐        1        ┌──────────────┐        N        ┌──────────────┐
│     User     │◀────────────────│   UserBook   │────────────────▶│     Book     │
├──────────────┤                 ├──────────────┤                 ├──────────────┤
│ id: UUID     │                 │ id: Long     │                 │ id: Long     │
│ email: String│                 │ status: Status│                 │ title: String│
│ password: Str│                 │ currentPage: int│               │ isbn: String │
│ role: Role   │                 │ startDate: Date│                │ pageCount    │
│ readingGoal  │                 │ endDate: Date │                 │ avgRating    │
└──────────────┘                 └──────────────┘                 │ coverUrl     │
       │ 1                                                          └──────┬───────┘
       │                                                                   │ N:M
       │        N                ┌──────────────┐                  ┌──────┴───────┐
       └────────────────────────▶│   Review     │                  │    Genre     │
                                 ├──────────────┤                  ├──────────────┤
                                 │ id: Long     │                  │ id: Long     │
                                 │ content: String│                │ name: String │
                                 │ rating: Double│                 └──────────────┘
                                 │ createdAt: Date│
                                 └──────────────┘
```

12. Maquettes UI/UX

12.1 Page d'Accueil (Landing)

```
┌──────────────────────────────────────────────────────────────┐
│  🚀 BookTracker            [Catalogue]  [Login] [S'inscrire]│
├──────────────────────────────────────────────────────────────┤
│                                                              │
│        Suivez vos lectures.                                  │
│        Découvrez de nouveaux livres.                         │
│                                                              │
│        [🔍 Rechercher un livre...                      ]     │
│        [Commencer gratuitement]                              │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│  📚 Livres les mieux notés                                   │
│  ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐                │
│  │ 📗     │ │ 📘     │ │ 📙     │ │ 📘     │                │
│  │ Titre1 │ │ Titre2 │ │ Titre3 │ │ Titre4 │                │
│  │ ⭐4.5  │ │ ⭐4.2  │ │ ⭐4.8  │ │ ⭐4.1  │                │
│  └────────┘ └────────┘ └────────┘ └────────┘                │
└──────────────────────────────────────────────────────────────┘
```

12.2 Dashboard Lecteur

```
┌──────────────────────────────────────────────────────────────┐
│  🚀 BookTracker   [Catalogue] [Biblio] [Stats]  👤 Profil    │
├─────────┬────────────────────────────────────────────────────┤
│ Menu    │  📊 Mon Tableau de Bord                            │
│         │                                                    │
│ 📚      │  ┌──────────┐  ┌──────────┐  ┌──────────┐         │
│ Biblio  │  │ 12 Livres│  │ 3 En cours│  │ 2 340 pg │         │
│         │  │ lus      │  │          │  │ lues     │         │
│ 📈      │  └──────────┘  └──────────┘  └──────────┘         │
│ Stats   │                                                    │
│         │  📖 En Cours de Lecture                            │
│ 💡      │  ┌──────────────────────────────────────┐          │
│ Reco    │  │ Dune — F. Herbert     [▓▓▓▓▓▓▓▓▓▓▓▓] 78%     │
│         │  │ 1984 — G. Orwell      [▓▓▓▓▓▓▓▓▓▓▓▓] 55%     │
│ ⚙       │  └──────────────────────────────────────┘          │
│ Param   │                                                    │
│         │  🎯 Objectif 2026 : 12/24 livres [▓▓▓▓▓▓▓▓▓▓▓▓] 50%  │
│         │                                                    │
│         │  💡 Recommandations pour vous                      │
│         │  ┌────────┐ ┌────────┐ ┌────────┐                │
│         │  │ 📗     │ │ 📘     │ │ 📙     │                │
│         │  │ Titre  │ │ Titre  │ │ Titre  │                │
│         │  └────────┘ └────────┘ └────────┘                │
└─────────┴────────────────────────────────────────────────────┘
```

13. Gestion des Risques

#	Risque	Probabilité	Impact	Mitigation
R1	Moteur de recommandation trop complexe	Élevée	Moyen	Utiliser un algorithme simple (filtrage par genre), pas de ML
R2	Surcharge de travail (projet solo, 2 mois)	Élevée	Élevé	Priorisation MoSCoW stricte, se concentrer sur le MVP
R3	Indisponibilité API Google Books	Faible	Moyen	Cache des résultats, fallback données locales
R4	Problèmes de performance	Faible	Faible	Cache Redis pour les requêtes lourdes
R5	Incompatibilités versions	Faible	Faible	Docker Compose pour uniformiser l'environnement

14. Livrables

#	Livrable	Format	Sprint
L1	Cahier de charges	Markdown / PDF	S0
L2	Code source Backend (Java / Spring Boot)	GitHub Repository	S1-S4
L3	Code source Frontend (React.js)	GitHub Repository	S1-S4
L4	Documentation API (Swagger / OpenAPI)	HTML auto-généré	S4
L5	Docker Compose — déploiement local	YAML	S4
L6	Rapport de tests unitaires	HTML (JUnit)	S4
L7	Rapport PFA écrit	PDF	S4
L8	Présentation soutenance	PowerPoint / PDF	S4
L9	Vidéo de démonstration (3–5 min)	MP4	S4

15. Annexes

Annexe A — API REST — Endpoints Principaux

Auth (`/api/auth`)

Méthode	Endpoint	Description	Auth
POST	`/api/auth/register`	Inscription	Non
POST	`/api/auth/login`	Connexion	Non
POST	`/api/auth/forgot-password`	Demander reset	Non

Méthode	Endpoint	Description	Auth
POST	`/api/auth/reset-password`	Réinitialiser mot de passe	Non

Users (`/api/users`)

Méthode	Endpoint	Description	Auth
GET	`/api/users/me`	Profil courant	READER
PUT	`/api/users/me`	Modifier profil	READER
DELETE	`/api/users/me`	Supprimer compte	READER
GET	`/api/users`	Lister utilisateurs	ADMIN
PUT	`/api/users/{id}/status`	Activer/Désactiver	ADMIN

Books (`/api/books`)

Méthode	Endpoint	Description	Auth
GET	`/api/books`	Lister / Rechercher	Non
GET	`/api/books/{id}`	Détail d'un livre	Non
POST	`/api/books`	Ajouter un livre	ADMIN
PUT	`/api/books/{id}`	Modifier un livre	ADMIN
DELETE	`/api/books/{id}`	Supprimer un livre	ADMIN
GET	`/api/books/popular`	Livres mieux notés	Non

Reading Tracker (`/api/tracking`)

Méthode	Endpoint	Description	Auth
GET	`/api/tracking/library`	Ma bibliothèque	READER
POST	`/api/tracking/library`	Ajouter un livre	READER
PUT	`/api/tracking/library/{id}`	Modifier statut / progression	READER
DELETE	`/api/tracking/library/{id}`	Retirer un livre	READER
GET	`/api/tracking/history`	Historique lectures	READER
GET	`/api/tracking/goal`	Objectif annuel	READER
PUT	`/api/tracking/goal`	Définir objectif	READER

Reviews (`/api/reviews`)

Méthode	Endpoint	Description	Auth

Méthode	Endpoint	Description	Auth
GET	`/api/reviews/book/{bookId}`	Avis d'un livre	Non
POST	`/api/reviews`	Créer un avis	READER
PUT	`/api/reviews/{id}`	Modifier un avis	READER
DELETE	`/api/reviews/{id}`	Supprimer un avis	READER / ADMIN

Recommendations (`/api/recommendations`)

Méthode	Endpoint	Description	Auth
GET	`/api/recommendations`	Mes recommandations (par genre)	READER
GET	`/api/recommendations/popular`	Livres populaires	Non

Statistics (`/api/stats`)

Méthode	Endpoint	Description	Auth
GET	`/api/stats/me`	Mes statistiques	READER
GET	`/api/stats/me/genres`	Répartition par genre	READER
GET	`/api/stats/me/timeline`	Timeline de lecture	READER

Annexe B — Variables d'Environnement

```
# ── auth-service ──────────────────────────────────────────
AUTH_DB_URL=jdbc:postgresql://auth-db:5432/auth_db
AUTH_DB_USERNAME=booktracker
AUTH_DB_PASSWORD=****
JWT_SECRET_KEY=****
JWT_EXPIRATION=86400000
SPRING_MAIL_HOST=smtp.gmail.com
SPRING_MAIL_USERNAME=****
SPRING_MAIL_PASSWORD=****

# ── book-service ──────────────────────────────────────────
BOOK_DB_URL=jdbc:postgresql://book-db:5432/book_db
BOOK_DB_USERNAME=booktracker
BOOK_DB_PASSWORD=****
GOOGLE_BOOKS_API_KEY=****

# ── tracker-service ───────────────────────────────────────
TRACKER_DB_URL=jdbc:postgresql://tracker-db:5432/tracker_db
TRACKER_DB_USERNAME=booktracker
TRACKER_DB_PASSWORD=****
BOOK_SERVICE_URL=http://book-service:8082
```

```
# ── reco-service ──────────────────────────────────────────
RECO_DB_URL=jdbc:postgresql://reco-db:5432/reco_db
RECO_DB_USERNAME=booktracker
RECO_DB_PASSWORD=****
TRACKER_SERVICE_URL=http://tracker-service:8083
BOOK_SERVICE_URL=http://book-service:8082
SPRING_REDIS_HOST=redis
SPRING_REDIS_PORT=6379

# ── api-gateway ───────────────────────────────────────────
AUTH_SERVICE_URL=http://auth-service:8081
BOOK_SERVICE_URL=http://book-service:8082
TRACKER_SERVICE_URL=http://tracker-service:8083
RECO_SERVICE_URL=http://reco-service:8084
JWT_SECRET_KEY=****
```

Annexe C — Structure du Projet Backend (Maven Multi-Modules)

```
booktracker/
├── pom.xml                             # Parent POM (gestion versions communes)
│
├── api-gateway/                        # Spring Cloud Gateway
│   ├── pom.xml
│   └── src/main/
│       ├── resources/application.yml    # Routes + filtres JWT
│       └── java/com/booktracker/gateway/
│           ├── GatewayApplication.java
│           └── filter/JwtAuthFilter.java
│
├── auth-service/                       # Port 8081
│   ├── pom.xml
│   └── src/main/java/com/booktracker/auth/
│       ├── AuthServiceApplication.java
│       ├── config/SecurityConfig.java
│       ├── controller/AuthController.java
│       ├── controller/UserController.java
│       ├── entity/User.java
│       ├── repository/UserRepository.java
│       ├── service/AuthService.java
│       ├── service/UserService.java
│       ├── dto/LoginRequest.java
│       ├── dto/RegisterRequest.java
│       └── security/JwtTokenProvider.java
│
├── book-service/                       # Port 8082
│   ├── pom.xml
│   └── src/main/java/com/booktracker/book/
│       ├── BookServiceApplication.java
│       ├── controller/BookController.java
│       ├── entity/Book.java
```

```
│       ├── entity/Author.java
│       ├── entity/Genre.java
│       ├── repository/BookRepository.java
│       ├── service/BookService.java
│       └── client/GoogleBooksClient.java  # OpenFeign externe
│
├── tracker-service/                     # Port 8083
│   ├── pom.xml
│   └── src/main/java/com/booktracker/tracker/
│       ├── TrackerServiceApplication.java
│       ├── controller/TrackingController.java
│       ├── controller/ReviewController.java
│       ├── entity/UserBook.java
│       ├── entity/Review.java
│       ├── repository/UserBookRepository.java
│       ├── repository/ReviewRepository.java
│       ├── service/TrackingService.java
│       ├── service/ReviewService.java
│       └── client/BookServiceClient.java  # OpenFeign → book-service
│
├── reco-service/                        # Port 8084
│   ├── pom.xml
│   └── src/main/java/com/booktracker/reco/
│       ├── RecoServiceApplication.java
│       ├── controller/RecommendationController.java
│       ├── controller/StatisticsController.java
│       ├── service/RecommendationService.java
│       ├── service/StatisticsService.java
│       ├── client/TrackerServiceClient.java  # OpenFeign → tracker-service
│       ├── client/BookServiceClient.java     # OpenFeign → book-service
│       └── config/RedisConfig.java
│
├── docker-compose.yml                   # Orchestration complète
└── .github/
    └── workflows/
        └── ci.yml                       # Build + tests toutes les push
```

docker-compose.yml — Structure principale

```yaml
services:
  api-gateway:
    build: ./api-gateway
    ports: ["8080:8080"]
    depends_on: [auth-service, book-service, tracker-service, reco-service]

  auth-service:
    build: ./auth-service
    ports: ["8081:8081"]
    depends_on: [auth-db]

  book-service:
```

```
    build: ./book-service
    ports: ["8082:8082"]
    depends_on: [book-db]

  tracker-service:
    build: ./tracker-service
    ports: ["8083:8083"]
    depends_on: [tracker-db]

  reco-service:
    build: ./reco-service
    ports: ["8084:8084"]
    depends_on: [reco-db, redis]

  auth-db:
    image: postgres:16
    environment: { POSTGRES_DB: auth_db }

  book-db:
    image: postgres:16
    environment: { POSTGRES_DB: book_db }

  tracker-db:
    image: postgres:16
    environment: { POSTGRES_DB: tracker_db }

  reco-db:
    image: postgres:16
    environment: { POSTGRES_DB: reco_db }

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  frontend:
    build: ./booktracker-frontend
    ports: ["3000:3000"]
```

Annexe D — Glossaire

Terme	Définition
JWT	JSON Web Token — standard de token d'authentification stateless
RBAC	Role-Based Access Control — contrôle d'accès basé sur les rôles
MoSCoW	Must / Should / Could / Won't — méthode de priorisation
MVP	Minimum Viable Product — produit minimum viable
CI/CD	Continuous Integration / Continuous Deployment

Terme	Définition
SPA	Single Page Application
Content-based filtering	Algorithme de recommandation basé sur le contenu (genres préférés)

Annexe E — Évolutions Futures (Hors Périmètre PFA)

Les fonctionnalités suivantes sont identifiées comme évolutions naturelles pour un **PFE** ou une version 2.0 :

Évolution	Description
Filtrage collaboratif	Recommandations basées sur les utilisateurs ayant des goûts similaires (ML)
Déploiement Cloud (AWS)	EKS, RDS, S3, CloudFront, Route 53, WAF
Orchestration Kubernetes	Helm Charts, HPA, déploiement blue/green
Pipeline CI/CD avancé	SonarQube, Trivy, tests E2E Cypress, déploiement automatisé
Authentification OAuth2	Connexion Google / GitHub
Communication asynchrone	RabbitMQ ou Kafka (notifications, événements lecture)
Service Discovery	Eureka Server (découverte automatique des services)
Recherche full-text	Intégration Elasticsearch
Monitoring centralisé	Prometheus + Grafana + ELK Stack
Internationalisation	Support anglais / français
Application mobile	React Native ou Flutter

> **Document validé par :** Mohamed Nacer Hammami **Date de validation :** 19/02/2026 **Version :** 1.0 (PFA — 2 mois) **Statut :** En vigueur